

1. Given an $N \times N$ matrix, Write a C Program to determine whether it is:

- a) Symmetric
- b) Upper triangular
- c) Lower triangular
- d) Diagonal matrix
- e) Identity matrix
- f) Display all applicable classifications.

our Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, a[10][10];
5     int sym=1, up=1, low=1, diag=1, id=1;
6
7     scanf("%d", &n);
8     for (i = 0; i < n; i++)
9         for (j = 0; j < n; j++)
10             scanf("%d", &a[i][j]);
11
12     for (i = 0; i < n; i++) {
13         for (j = 0; j < n; j++) {
14             if (a[i][j] != a[j][i]) sym = 0;
15             if (i > j && a[i][j] != 0) up = 0;
16             if (i < j && a[i][j] != 0) low = 0;
17             if (i != j && a[i][j] != 0) diag = 0;
18             if (i == j && a[i][j] != 1) id = 0;
19             if (i != j && a[i][j] != 0) id = 0;
20         }
21     }
22
23     if (sym) printf("Symmetric\n");
24     if (up) printf("Upper Triangular\n");
25     if (low) printf("Lower Triangular\n");
26     if (diag) printf("Diagonal\n");
27     if (id) printf("Identity\n");
28     if (!sym && !up && !low && !diag && !id) printf("None\n");
29
30     return 0;
31 }
```

Input

```
3
1 0 0
0 1 0
0 0 1
```

Output

```
Symmetric
Upper Triangular
Lower Triangular
Diagonal
Identity
```

2. Given an array of integers, write a C program that:
- Determines whether the array is sorted.
 - If sorted, performs Binary Search.
 - Otherwise, performs Linear Search.
 - Displays the search method used and the number of comparisons performed.

Your Code

```
1 #include <stdio.h>
2 void main() {
3     int n, key;
4     scanf("%d", &n);
5     int a[n];
6     for(int i=0;i<n;i++) scanf("%d",&a[i]);
7     scanf("%d",&key);
8
9     int cmp=0, sorted=1;
10
11     for(int i=0;i<n-1;i++){
12         cmp++;
13         if(a[i]>a[i+1]) { sorted=0; break; }
14     }
15     int idx=-1, scmp=0;
16
17     if(sorted){
18         int l=0,h=n-1;
19         while(l<=h){
20             int m=(l+h)/2;
21             scmp++;
22             if(a[m]==key){ idx=m; break; }
23             scmp++;
24             if(key<a[m]) h=m-1;
25             else l=m+1;
26         }
27         printf("Binary Search\n");
28     } else {
29         for(int i=0;i<n;i++){
30             scmp++;
31             if(a[i]==key){ idx=i; break; }
32         }
33         printf("Linear Search\n");
34     }
35     printf("%d\n", idx);
36     printf("%d\n", cmp+scmp);
37 }
```

Input

```
5
1 2 3 4 5
4
```

Output

```
Binary Search
3
7
```

3. Store marks of N students in M subjects using a two-dimensional array. Generate a report containing:

- a) Student topper
- b) Subject topper
- c) Class average
- d) Subject-wise average
- e) Students scoring above the overall average
- f) Rank list without modifying the original data

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int N, M;
5     scanf("%d %d", &N, &M);
6
7     int a[20][20], total[20], sTot[20];
8     int i, j;
9
10    for (i = 0; i < N; i++) {
11        total[i] = 0;
12        for (j = 0; j < M; j++) {
13            scanf("%d", &a[i][j]);
14            total[i] += a[i][j];
15        }
16    }
17
18    for (j = 0; j < M; j++) {
19        sTot[j] = 0;
20        for (i = 0; i < N; i++) sTot[j] += a[i][j];
21    }
22
23    int topper = 0;
24    for (i = 1; i < N; i++) if (total[i] > total[topper]) topper = i;
25
26    int subjTop[20];
27    for (j = 0; j < M; j++) {
28        subjTop[j] = 0;
29        for (i = 1; i < N; i++)
30            if (a[i][j] > a[subjTop[j]][j]) subjTop[j] = i;
31    }
32
33    int grand = 0;
34    for (i = 0; i < N; i++) grand += total[i];
35    double classAvg = (double)grand / (N * M);
```

Input

```
4 3
80 90 70
60 75 85
90 88 92
```

Output

```
Student topper: Student 3
Subject topper:
S1 -> Student 3
S2 -> Student 1
S3 -> Student 3
Class average: 77.08
```

4. Given N strings, write a C program to:

- a) Sort them lexicographically.
- b) Find the longest and shortest strings.
- c) Count duplicate strings.
- d) Display strings beginning with vowels.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4 int vowel(char c){
5     c = tolower((unsigned char)c);
6     return c=='a' || c=='e' || c=='i' || c=='o' || c=='u';
7 }
8 int main() {
9     int N;
10    char s[50][50];
11    printf("N: ");
12    scanf("%d", &N);
13    for(int i=0;i<N;i++) scanf("%49s", s[i]);
14    for(int i=0;i<N-1;i++)
15        for(int j=0;j<N-1-i;j++)
16            if(strcmp(s[j], s[j+1]) > 0){
17                char t[50];
18                strcpy(t, s[j]);
19                strcpy(s[j], s[j+1]);
20                strcpy(s[j+1], t);
21            }
22
23    printf("\nSorted:\n");
24    for(int i=0;i<N;i++) printf("%s\n", s[i]);
25    int lo=0, sh=0;
26    for(int i=1;i<N;i++){
27        if(strlen(s[i]) > strlen(s[lo])) lo=i;
28        if(strlen(s[i]) < strlen(s[sh])) sh=i;
29    }
30    printf("\nLongest: %s\nShortest: %s\n", s[lo], s[sh]);
31
32
33    int dup=0;
34    for(int i=0;i<N-1;i++)
35        if(strcmp(s[i], s[i+1])==0) dup++;
36    printf("\nDuplicate count: %d\n", dup);
37    printf("\nVowel-start strings:\n");
38    int f=0;
```

Input

```
5
apple Orange apple Banana igloo
```

Output

```
N:
Sorted:
Banana
Orange
apple
apple
```